

TourPlanner – Finales Protokoll

Team: Lukas Fink, Franziska Brandfellner

Git repository: <https://github.com/lukifi1/SWEN2>

1. Technische Schritte und Entscheidungen

1.1 Projektziel

TourPlanner ist eine Two-Tier-Webanwendung zur Planung, Verwaltung und Auswertung persönlicher Touren wie Wandern, Radfahren, Laufen oder Urlaubstouren. Benutzer können sich registrieren, einloggen, eigene Touren anlegen, Routen berechnen lassen, Tour-Logs erfassen, Touren durchsuchen, Tourdaten importieren/exportieren und ein Statistik-Dashboard verwenden.

Die Anwendung besteht aus einem Angular-Frontend und einem Java/Spring-Boot-Backend. Persistente Daten werden in PostgreSQL gespeichert. OpenRouteService liefert Geocoding und Directions. Leaflet und OpenStreetMap werden für die Kartendarstellung verwendet.

1.2 Entwicklung von der Zwischenabgabe zur Finalabgabe

In der Zwischenabgabe lag der Fokus auf dem Frontend. Touren und Logs wurden lokal im Angular-State gehalten und das Backend war nur als leeres Spring-Boot-Gerüst vorhanden. Für die Finalabgabe wurde daraus eine echte Two-Tier-Anwendung: Das Frontend kommuniziert über HTTP/JSON mit einer Spring-Boot-REST-API, die Authentifizierung, Businesslogik, Persistenz, Suche, Import/Export, Bildspeicherung und Statistiken übernimmt.

Das Frontend wurde auf eine klarere Struktur mit core, features und shared umgestellt. Das Backend wurde in presentation, business und dal gegliedert. Dadurch sind UI, REST-Schnittstelle, Fachlogik und Persistenz voneinander getrennt.

1.3 Backend-Architektur

Das Backend folgt einer Layered Architecture.

presentation: Enthält REST-Controller und DTOs. Controller nehmen Requests entgegen, validieren DTOs, lesen den authentifizierten Benutzer aus dem Security Context und delegieren an Services. Controller greifen nicht direkt auf Repositories zu.

business: Enthält die fachliche Logik. Wichtige Klassen sind TourService, TourLogService, TourSearchService, StatsService, AuthService, ImportExportService, ImageStorageService, ComputedAttributesService, JwtService und OpenRouteServiceClient. Hier werden Berechtigungen, berechnete Attribute, Suchlogik, Import/Export und externe ORS-Kommunikation behandelt.

dal: Enthält Spring-Data-Repositories für User, Tour und TourLog. Datenbankzugriffe sind benutzerbezogen, damit ein Benutzer nur seine eigenen Touren und Logs sieht oder verändert.

Fehler werden über eine eigene Business-Exception-Hierarchie behandelt. Technische Persistenzfehler werden nicht direkt nach oben durchgereicht, sondern in Exceptions übertragen. GlobalExceptionHandler wandelt diese in HTTP-Fehlerantworten um.

1.4 Frontend-Architektur

Das Frontend folgt dem MVVM-Gedanken. core enthält Models, API-Services und Auth-Infrastruktur. features enthält die fachlichen UI-Bereiche für Auth, Tours und Stats. shared enthält wiederverwendbare UI-Komponenten wie ActionButton, StatCard und BarChart.

Models und API-Services liegen unter core/models und core/api. Dazu gehören TourApiService, TourLogApiService, AuthApiService, DataApiService und StatsApiService. core/auth enthält AuthService, AuthInterceptor und AuthGuard für JWT-Handling und geschützte Routen.

ViewModels wie AuthViewModel, ToursViewModel, TourFormViewModel, TourLogsViewModel und StatsViewModel halten UI-Zustand mit Angular Signals, computed UI-State und Commands für die Views bereit. AuthViewModel kapselt Login/Register-State, TourFormViewModel kapselt Formularlogik, Location-Autocomplete, Bild-Upload und DTO-Erzeugung. Komponenten bleiben dadurch stärker auf Darstellung und Binding fokussiert.

1.5 Datenfluss

Der typische Ablauf ist:

Benutzerinteraktion in einer Angular-Komponente -> Command im ViewModel -> API-Service -> HTTP-Request -> Spring Controller -> Business Service -> Repository oder externer Dienst -> DTO Response -> Aktualisierung der Signals -> aktualisierte View.

Bei Login/Register ruft das Frontend die Auth-Endpunkte auf. Nach erfolgreichem Login wird das JWT gespeichert. Der AuthInterceptor hängt das Token an geschützte Requests an. Im Backend validiert ein JWT-Filter das Token und stellt den Benutzernamen als Authentication bereit.

1.6 OpenRouteService und Leaflet

Beim Erstellen oder Aktualisieren einer Tour werden Startort, Zielort und Transporttyp an OpenRouteService übergeben. Das Backend berechnet daraus Distanz, geschätzte Dauer und Routengeometrie. Die Geometrie wird gespeichert und im Frontend mit Leaflet als Route dargestellt.

Leaflet wurde gewählt, weil es leichtgewichtig ist, gut mit OpenStreetMap funktioniert und für Routenanzeige im Browser geeignet ist. Ein Problem war, dass Leaflet Karten falsch rendert, wenn der Container beim Initialisieren noch nicht sichtbar oder nicht korrekt dimensioniert ist. Die Lösung war die Initialisierung in passenden Angular-Lifecycle-Hooks und ein invalidateSize nach dem Rendern bzw. nach Tourwechseln.

1.7 Import/Export und Bildspeicherung

ImportExportService hängt nur vom Interface ImportExportStrategy ab (Strategy-Abstraktion). GpxImportExportStrategy ist die aktive Implementierung; JsonImportExportStrategy ist zusätzlich vorhanden und testbar. Neue Formate können dadurch ergänzt werden, ohne den Service umzubauen.

Bilder werden nicht als BLOBs in der Datenbank gespeichert, sondern über ImageStorageService im Dateisystem abgelegt. In der Datenbank wird nur der Dateiname bzw. Pfad referenziert. Dadurch bleibt die Datenbank weniger belastet.

2. Probleme, Entscheidungen und gewählte Lösungen

2.1 Von lokalem State zu echter Persistenz

Die größte Änderung gegenüber der Zwischenabgabe war der Wechsel von lokal gehaltenen Beispieldaten zu einer persistenten Backend-Lösung. Das resultiert in einer klaren Trennung zwischen Angular API-Services, Spring REST-Controllern, Business-Services und Repositories.

2.2 Authentifizierung und Benutzertrennung

Benutzer dürfen nur ihre eigenen Touren und Logs sehen. Die Lösung ist stateless JWT-Authentifizierung. Controller übergeben den authentifizierten Benutzernamen an Services. Services laden Daten immer benutzerbezogen und werfen fachliche Fehler, wenn Ressourcen nicht existieren oder nicht zum Benutzer gehören.

2.3 Externe ORS-Abhängigkeit

OpenRouteService ist notwendig für reale Routenberechnung, kann aber durch fehlende API-Keys, ungültige Orte, Netzwerkfehler oder unerwartete Antworten fehlschlagen. Diese Logik wird im OpenRouteServiceClient gekapselt und als RouteCalculationException weitergegeben. In Tests wird ORS gemockt, damit die Tests stabil bleiben.

2.4 Full-Text-Suche über mehrere Datenquellen

Die Suche soll Tourfelder, Log-Kommentare und berechnete Werte wie Popularity und Child-friendliness durchsuchen. Die Lösung ist ein kombinierter Suchtext pro Tour. Dieser enthält Tourdaten, Logdaten und Textlabels berechneter Attribute.

2.5 Berechnete Attribute

Popularity und Child-friendliness werden aus Logs, Schwierigkeit, Dauer und Distanz abgeleitet. Sie werden nach Log-Änderungen neu berechnet. Dadurch bleiben Detailansicht, Suche und Statistik konsistent.

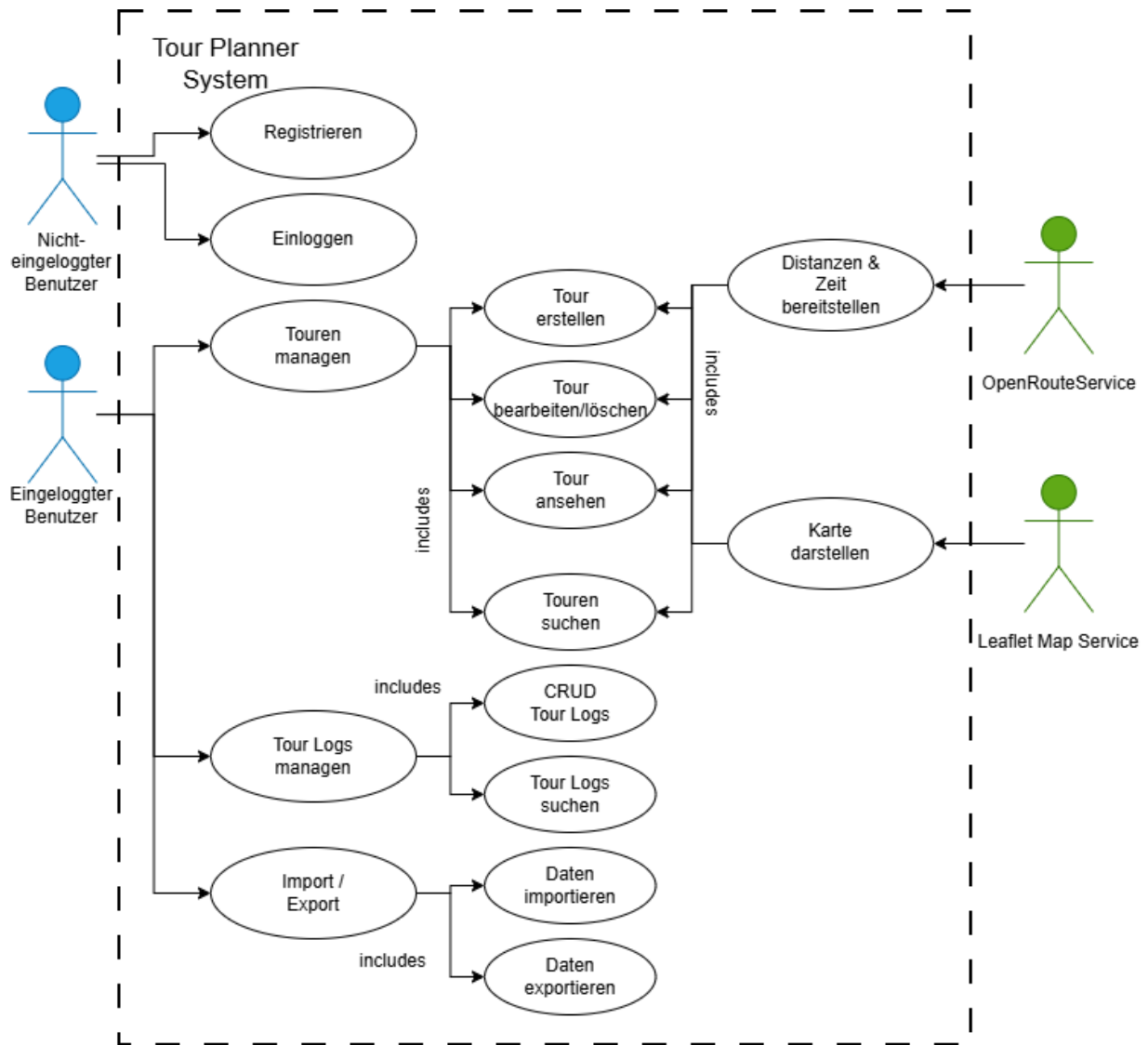
3. Implementierte Features

- Registrierung und Login mit JWT
- Benutzerbezogene Tourverwaltung: erstellen, anzeigen, bearbeiten und löschen
- Routenberechnung über OpenRouteService inklusive Distanz, Dauer und Geometrie
- Kartendarstellung mit Leaflet
- Tour-Log-Verwaltung mit Datum, Kommentar, Schwierigkeit, Distanz, Zeit und Bewertung
- Berechnete Attribute: Popularity und Child-friendliness
- Full-Text-Suche über Touren, Logs und berechnete Textlabels
- GPX Import und Export
- Bild-Upload und Auslieferung von Bildern
- Statistik-Dashboard mit Karten, Diagrammen und Popularitätsranking

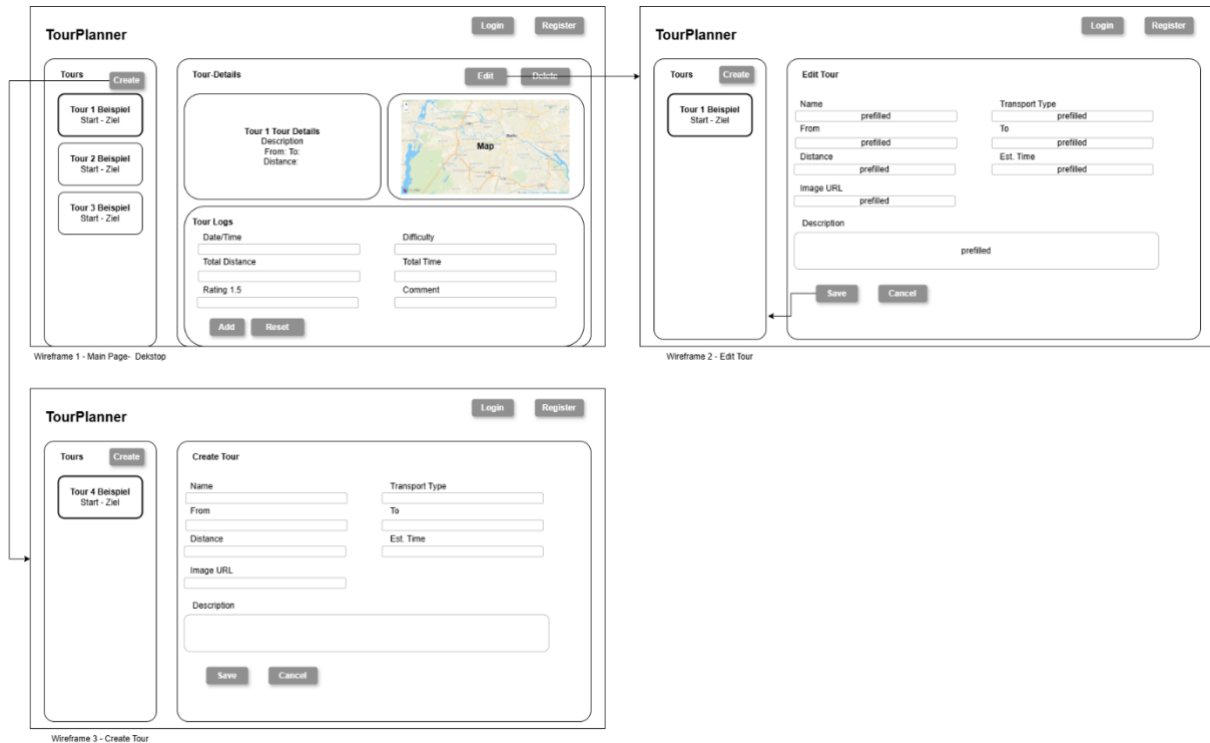
- Client- und serverseitige Validierung

- Responsive UI-Struktur (Für kleinere Viewports werden Navigation, Listen, Details und Formulare einspaltig dargestellt.)

4. UML Use-Case-Diagramm



5. UI-Flow und Wireframes



Login

Benutzer gibt Username und Passwort ein. Bei Erfolg wird das JWT gespeichert und zur Tourübersicht navigiert. Bei Fehler wird eine Fehlermeldung angezeigt.

Registrierung

Benutzer legt Username und Passwort an. Nach erfolgreicher Registrierung kann er sich anmelden.

Tourübersicht

Liste der vorhandenen Touren mit Suchfeld und Aktionen. Der Benutzer kann eine Tour auswählen, eine neue Tour erstellen, Daten importieren/exportieren und zur Statistik wechseln.

Tour erstellen/bearbeiten

Formular mit Name, Beschreibung, Start, Ziel, Transporttyp und optionalem Bild. Beim Speichern ruft das Backend ORS auf und ergänzt Distanz, Dauer und Route.

Tourdetails

Anzeige der Tourdaten, berechneten Attribute, Bild, Karte mit Route und zugehörigen Logs. Aktionen: Bearbeiten, Löschen, Log hinzufügen, Logs bearbeiten oder Logs löschen.

Suche

Benutzer gibt Suchbegriff ein. Die Trefferliste aktualisiert sich anhand der Backend-Suche über Tourfelder, Log-Kommentare und berechnete Labels.

Statistik-Dashboard

Zusammenfassungskarten für Anzahl Touren/Logs, geplante und geloggte Distanz, geloggte Zeit und Durchschnittsbewertung; Diagramme für Schwierigkeit, Bewertung, Transporttyp und Distanz über Zeit; Tabelle mit beliebtesten Touren.

6. Anwendungsarchitektur und UML

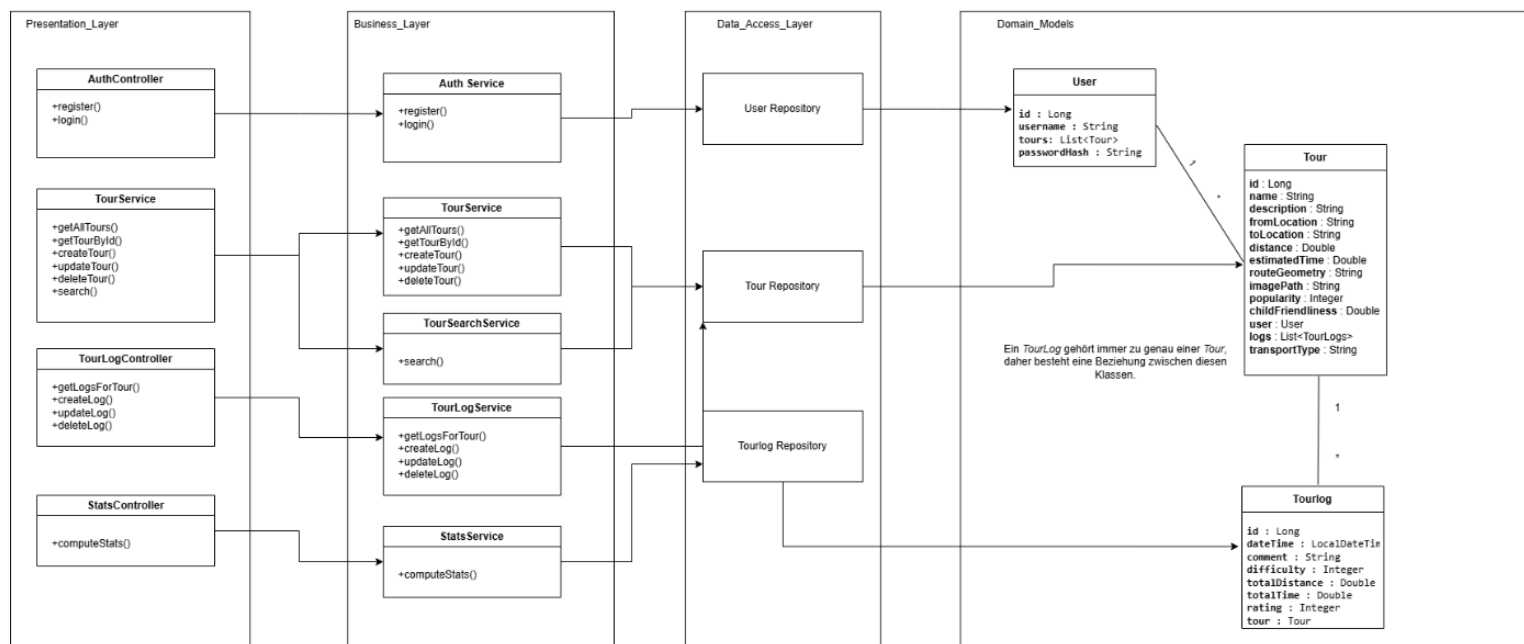
6.1 Architekturübersicht

Angular Frontend -> HTTP/JSON -> Spring Boot REST API -> Business Services -> Spring Data Repositories -> PostgreSQL.

Zusätzlich ruft das Backend OpenRouteService auf. Das Frontend lädt Kartenkacheln von OpenStreetMap und zeigt gespeicherte Routengeometrien mit Leaflet an.

6.2 Klassendiagramm

Class diagram



Der Presentation Layer besteht aus REST-Controllern, die HTTP-Requests entgegennehmen und an Services delegieren.

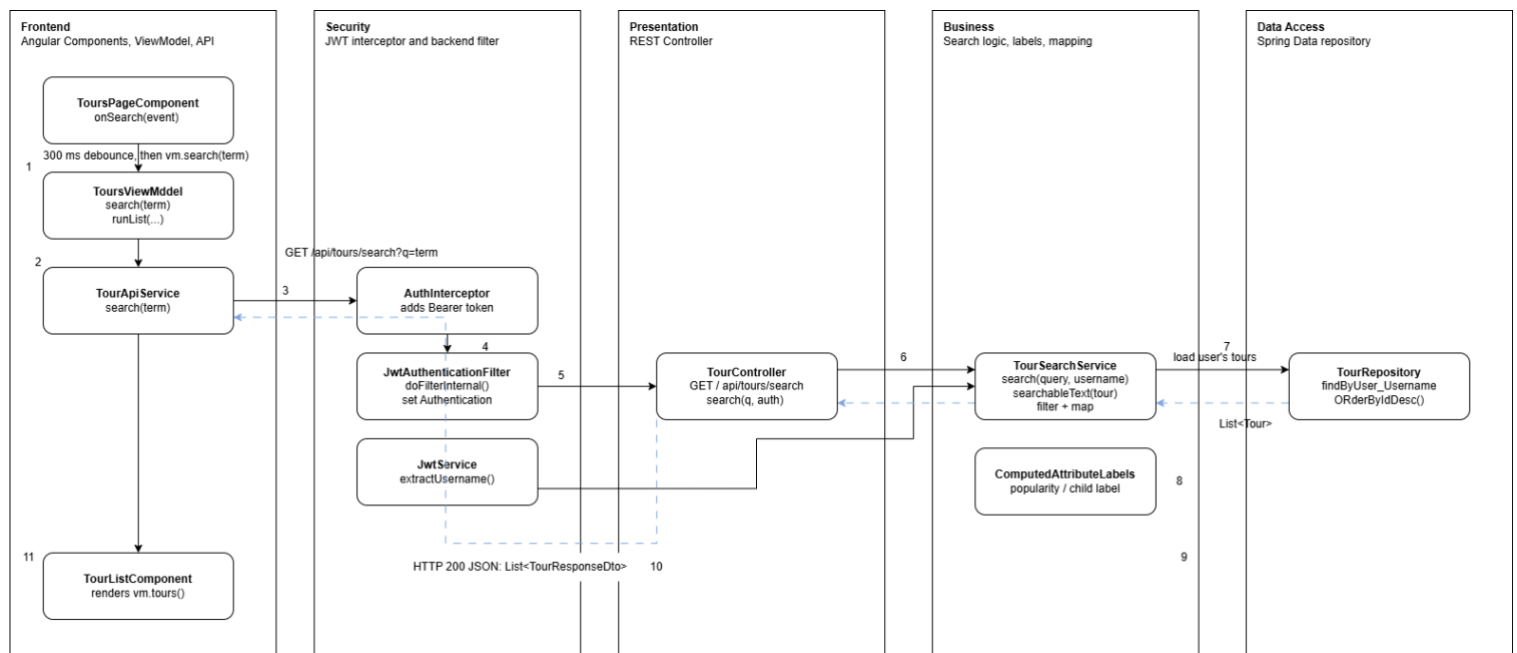
Der Business Layer enthält die fachliche Logik, z. B. Tourverwaltung, Tour-Logs, Authentifizierung, Suche und Statistikberechnung.

Der Data Access Layer besteht aus Spring-Data-Repositories, welche den Zugriff auf die JPA-Entities User, Tour und TourLog kapseln.

DTOs (Data Transfer Objects) definieren die Daten, die zwischen Frontend und Backend über die REST-API ausgetauscht werden. Sie trennen die externe API-Struktur von den internen JPA-Entities. Zum Beispiel enthält `TourCreateDto` die Eingabedaten zum Erstellen oder Bearbeiten einer Tour, während `TourResponseDto` die Tourdaten enthält, die an das Frontend zurückgegeben werden. Weitere DTOs sind `TourLogCreateDto`, `TourLogResponseDto`, `LoginRequest`, `RegisterRequest`, `AuthResponse` und `StatsDto`.

Das Frontend ist nach dem **MVVM-Prinzip** (Model-View-ViewModel) aufgebaut. Angular Components bilden die Views, z. B. `LoginComponent`, `RegisterComponent`, `ToursPageComponent`, `TourDetailComponent`, `TourLogsComponent` und `StatsPageComponent`. ViewModels wie `AuthViewModel`, `ToursViewModel`, `TourFormViewModel`, `TourLogsViewModel` und `StatsViewModel` halten den signal-basierten UI-State und stellen Aktionen für die Views bereit. Dadurch kennen komplexe Komponenten keine API-Details für Login/Register, Import/Export, Autocomplete, Upload oder DTO-Mapping. API-Services wie `TourApiService`, `TourLogApiService`, `AuthApiService` und `StatsApiService` kapseln die HTTP-Kommunikation mit dem Backend.

7. Sequence Diagram für Full-Text Search



1. Benutzer gibt einen Suchbegriff im Angular-Frontend ein.
2. `ToursViewModel` ruft `TourApiService.search(query)` auf.
3. `AuthInterceptor` hängt das JWT an den HTTP-Request an.
4. Backend-JWT-Filter validiert das Token und erstellt Authentication.
5. `TourController` empfängt `GET /api/tours/search?q=...` und übergibt `query` sowie `auth.getName()` an `TourSearchService`.

6. TourSearchService lädt alle Touren des Benutzers über `TourRepository.findByUser_UsernameOrderByIdDesc(username)`.
7. Für jede Tour wird ein Suchtext aus Name, Beschreibung, Start, Ziel, Transporttyp, Distanz, geschätzter Zeit, Popularity, ChildFriendliness, deren Labels sowie allen Log-Kommentaren und Logwerten aufgebaut.
8. TourSearchService filtert Touren, deren Suchtext den normalisierten Suchbegriff enthält.
9. Gefundene Touren werden mit TourMapper zu TourResponseDto gemappt.
10. TourController sendet die Ergebnisliste als JSON zurück.
11. ToursViewModel aktualisiert seine Signals; die Tourliste rendert die Treffer.

8. Design Patterns

Strategy Pattern: ImportExportStrategy definiert Import/Export-Operationen. GpxImportExportStrategy ist die aktive Implementierung, JsonImportExportStrategy ist zusätzlich vorhanden. ImportExportService hängt nur von der Abstraktion ab.

Repository Pattern: UserRepository, TourRepository und TourLogRepository kapseln Datenbankzugriffe über Spring Data JPA.

Dependency Injection: Spring und Angular DI stellen Services, Repositories, Controller und ViewModels bereit. Dadurch werden Klassen testbarer und weniger stark gekoppelt.

DTO und Mapper: REST-DTOs trennen externe API-Verträge von internen JPA-Entities. TourMapper und TourLogMapper wandeln Entities in Response-DTOs um.

MVVM im Frontend: Views binden an ViewModel-Signals. ViewModels halten Zustand und Commands, API-Services übernehmen HTTP-Kommunikation.

9. Bibliotheksentscheidungen und Lessons Learned

Angular 19 wurde gewählt, weil es moderne Standalone Components, TypeScript, Routing, Forms und gute Testwerkzeuge bietet.

Spring Boot wurde gewählt, weil damit REST APIs, Security, Validation, JPA und Testing in einem konsistenten Java-Stack umgesetzt werden können.

PostgreSQL wurde gewählt, weil relationale Tour-, Benutzer- und Logdaten gut modellierbar sind und PostgreSQL als robuste Datenbank geeignet ist.

Spring Data JPA/Hibernate reduziert Boilerplate für Repositories und erlaubt klare Entity-Beziehungen.

Spring Security mit JWT wurde gewählt, weil die API zustandslos bleiben kann und das Angular-Frontend Tokens einfach mitsenden kann.

Leaflet wurde gewählt, weil es für Webkarten leichtgewichtig und für OpenStreetMap gut geeignet ist.

OpenRouteService wurde gewählt, weil reale Geocoding- und Routingdaten benötigt werden und ORS passende Profile für verschiedene Transportarten anbietet.

Außerdem waren diese Dienste/Bibliotheken in der Angabe vorgegeben.

Log4j2 wurde verwendet, um Logging zentral zu konfigurieren und die Projektanforderung zu erfüllen.

Lessons Learned: Die größten Integrationspunkte waren Authentifizierung, externe API-Fehler, saubere Benutzertrennung, Leaflet-Lifecycle-Probleme und das Aufteilen von UI-Zustand in ViewModels. Tests mit Mocks waren wichtig, damit externe Services und Datenbankdetails die Fachlogiktests nicht instabil machen.

10. Unit-Testing-Entscheidungen

Im Backend sind 77 @Test-Methoden vorhanden. Die Tests konzentrieren sich auf fachlich kritische Pfade, weil Fehler dort Datenverlust, Sicherheitsprobleme oder falsche Berechnungen verursachen könnten.

AuthService und JwtService werden getestet, weil Registrierung, Login, Passwortprüfung und Token-Erstellung sicherheitskritisch sind.

TourService wird getestet, weil Touren zentrale Domänenobjekte sind. Kritisch sind Create/Update/Delete, Benutzerzuordnung, ORS-Routingdaten und Fehlerfälle bei nicht vorhandenen oder fremden Ressourcen.

TourLogService wird getestet, weil Logs die Grundlage für Bewertungen, Statistik und berechnete Attribute sind.

TourSearchService wird getestet, weil die Suche mehrere Datenquellen kombiniert: Tourfelder, Log-Kommentare und Labels berechneter Attribute.

StatsService wird getestet, weil das Statistik-Dashboard als Unique Feature korrekte Aggregationen benötigt: Summen, Durchschnittswerte, Verteilungen, Transporttypen, Distanz über Zeit und Popularitätsranking.

ComputedAttributes und ComputedAttributeLabels werden getestet, weil sie direkt in UI, Suche und Statistik einfließen.

ImportExportService und ImportExportStrategy-Implementierungen werden getestet, weil Import/Export Daten zwischen Systemen bewegt und Fehler zu Datenverlust oder falsch importierten Touren führen können.

OpenRouteServiceClient und TransportProfileResolver werden getestet, weil externe API-Kommunikation und Profil-Mapping fehleranfällig sind. Tests prüfen Mapping, Fehlerbehandlung und Antwortverarbeitung ohne echte ORS-Aufrufe.

ImageStorageService wird getestet, weil Dateinamen, Pfade und Dateisystemzugriffe sicher gehandhabt werden müssen.

AuthFlowIntegrationTest prüft zentrale Authentifizierungsabläufe über die REST-Schicht mit MockMvc. Dadurch wird nicht nur isolierte Logik, sondern auch das Zusammenspiel von Controller, Security und Service geprüft.

Frontend-Tests sind als Angular-Specs vorhanden. Sie prüfen API-Services, Auth-Session-State, ViewModel-Commands, Autocomplete, Bild-Upload, DTO-Mapping und abgeleiteten computed

UI-State. Der Backend-Fokus liegt zusätzlich auf fachlich kritischen Services, Security-Flows und Integrationspunkten.

11. Unique Feature: Statistics Dashboard

Das Unique Feature der Anwendung ist das Statistik-Dashboard. Es vereint alle Touren und Logs eines Benutzers und stellt wichtige Kennzahlen übersichtlich dar.

Angezeigt werden Gesamtanzahl der Touren und Logs, geplante Tourdistanz, tatsächlich geloggte Distanz, geloggte Zeit, durchschnittliche Bewertung, Schwierigkeitsverteilung, Bewertungsverteilung, Touren nach Transporttyp, geloggte Distanz über Zeit und ein Ranking der beliebtesten Touren mit Child-friendliness-Label.

Das Feature verwendet mehrere Teile der Anwendung gleichzeitig: Persistenz, Logs, berechnete Attribute, Aggregationslogik, DTOs, API-Service, ViewModel und wiederverwendbare Chart-Komponenten und macht sie übersichtlich darstellbar.

12. Zeiterfassung

Lukas: Repository angelegt und initialer Projektstand vorbereitet, Projektsetup, README, erster Angular-Frontend-Draft und Grundstruktur | ca. 6 h | Initial commit, initial setup, first draft of frontend, README commits

Lukas: Frontend-Design verbessert, Komponentenverwendung überarbeitet, ActionButton-Styling, Leaflet-Karte eingebunden | ca. 5 h | Frontend design/component usage, colors, leaflet map

Franziska: Zwischenprotokoll erstellt und README aktualisiert | ca. 5 h | Added protocol and updated README

Franziska: Angular-State für showTourForm auf Signal umgestellt | ca. 1 h | introduce signal for showTourForm

Franziska: editMode auf Signal umgestellt und Angular Template Control Flow modernisiert | ca. 3 h | convert editMode to signal, migrate to new control flow syntax

Lukas: Koordinatenunterstützung für Tourdaten ergänzt | ca. 2 h | support for saving longitude and latitude

Lukas: Backend-Struktur entworfen, Spring Security Dependency ergänzt, erster Datenbankverbindungsversuch | ca. 4 h | backend outline, first backend draft, db connection not working yet

Lukas: Backend final neu strukturiert: Dependencies, Properties, Logging, REST-Client-Konfiguration, Security Config, JWT-Filter, Exception-Hierarchie, GlobalExceptionHandler, Entities und Tour-DTOs | ca. 8 h | mehrere Backend-Konfigurations- und Strukturcommits

Lukas: DTOs, Mapper, benutzerbezogene Repositories und berechnete Attribute implementiert | ca. 7 h | auth/log/error/image/import/stats DTOs, repositories, mappers, computed attributes

Lukas: Zentrale Business-Services implementiert: Auth, CurrentUser, JWT, OpenRouteServiceClient, TourService, TourLogService, TourSearchService, StatsService und ImageStorageService | ca. 10 h | Service-Implementierungen und ORS/Image/Search/Stats

Lukas: REST-Controller, Import/Export-Service und Helper ergänzt, WebConfig bereinigt, Backend-Testabdeckung stark erweitert | ca. 9 h | auth/tour/log/stats/image/import-export controllers, import/export service, backend tests

Franziska: Angular-Konfiguration und globales Styling aktualisiert | ca. 1,5 h | Angular-Konfiguration, globale Styles und index.html angepasst, damit die neue Frontend-Struktur sauber eingebunden ist.

Franziska: Frontend-Core-Struktur erstellt | ca. 3 h | Gemeinsame Models, API-Clients und Formatierungslogik für Tourdaten angelegt. Dadurch wurde die Kommunikation mit dem Backend zentralisiert und wiederverwendbar gemacht.

Franziska: Authentifizierung im Frontend umgesetzt | ca. 3,5 h | Auth-Service, Auth-Interceptor, Auth-Guard sowie Login- und Register-Seiten erstellt. Dadurch können geschützte Routen und JWT-basierte Requests verwendet werden.

Franziska: Signal-basiertes Tours ViewModel implementiert | ca. 2,5 h | ViewModel für Touren erstellt, um Ladezustand, ausgewählte Tour, Suchergebnisse, Fehler und UI-Aktionen zentral zu verwalten.

Franziska: Tour-Übersicht, Liste und Detailansicht umgesetzt | ca. 5 h | Neue Tour-Seite mit Tourliste und Detailkomponente erstellt. Anzeige von Tourdaten, Auswahlzustand und Detailinformationen an die neue Core/API-Struktur angebunden.

Franziska: Tour-Formular und Tour-Logs umgesetzt | ca. 4,5 h | Komponenten zum Erstellen und Bearbeiten von Touren sowie zum Verwalten von Tour-Logs erstellt. Formularlogik, Validierung und API-Anbindung umgesetzt.

Franziska: Statistik-Feature und Shared UI Components erstellt | ca. 4 h | Statistik-Seite mit ViewModel angebunden und wiederverwendbare UI-Komponenten wie ActionButton, BarChart und StatCard ergänzt.

Franziska: App-Routing und Root Layout aktualisiert | ca. 2 h | Routing auf Login, Register, Tours und Stats angepasst. Root-Komponente und Layout auf die neue Feature-Struktur umgestellt.

Franziska: Alte Frontend-Struktur entfernt und README aktualisiert | ca. 1,5 h | Veraltete Komponenten, alte Services und alte Models entfernt. README an die neue Projektstruktur angepasst.

Lukas: MVVM-Struktur im Frontend nachgeschärft | ca. 6 h | AuthViewModel ergänzt, AuthService auf Session-State reduziert, Import/Export in ToursViewModel verschoben, TourFormViewModel für Autocomplete, Bild-Upload und DTO-Mapping erstellt, direkte Subscriptions aus Komponenten reduziert und computed UI-State erweitert.

Gemeinsame finale Protokollüberarbeitung, fehlende Kapitel ergänzt und Text für Architektur, Tests, Patterns, Unique Feature und Zeiterfassung vorbereitet | ca. 3 h | Protokoll-WIP2 und Abgleich mit Abgabecheckliste

Gesamt:

Frontend: ca. 50,5 h

Backend: ca. 38 h

Protokoll/Dokumentation: ca. 8 h

Gesamt: ca. 96,5 h